

PHASE 1 · AUDIT + RESEARCH + ARCHITECTURE

AI Live Character Platform

Findings, verified provider research, proposed three-plane architecture, security and privacy model, and phased implementation plan — prepared for approval before any major build activity begins.

Architecture & Engineering Review

Covers control, media and compute planes · AI Brain tool governance · GPU worker fleet · admin control plane · Nigeria (NDPA) privacy considerations

SEPTEMBER 2026 · APPROVAL GATE DELIVERABLE

Table of Contents

1. Executive Summary	3
Workspace inventory	4
Audit conclusions against the nine required classifications	4
3. Verified Provider Research	5
Critical finding 1 - MiniMax H3 is real, and its license decides the architecture	6
Critical finding 2 - mobile virtual cameras are impossible; the phone-to-phone adapter is the honest path	6
Critical finding 3 - hosting terms shape the launch decision	7
4. Proposed Three-Plane Architecture	7
Control plane	8
Media plane	8
Compute plane	8
Why the separation matters	9
5. Technology Decisions and Rationale	9
6. Cost and Free-Tier Analysis	10
7. AI Brain Architecture	12
What the Brain can never do	13
Model independence	13
8. GPU Worker System	13
Lifecycle, selection, and the sleep system	14
Failure recovery	15
Session state machine	15
Job system	15
Media storage	15
Output adapters	15
Web application	16
Mobile application	17
3D usage policy	17
11. Security Model	17
Roles and administration	18
Nigeria Data Protection Act posture	18
Privacy data map and retention	19

Consent system	19
Cookies, legal pages, and honest claims	20
13. Accessibility Plan (WCAG 2.2 AA)	20
14. Testing Plan	21
Topology and environments	22
Environment variables and secrets	22
Operations	22
16. Implementation Phases	23
Honest limitation register	24
Risk register	24
18. Approval Gate and Open Decisions	25

1. Executive Summary

This document is the Phase-1 deliverable mandated by the build specification: a full audit of the current state, verified research on every external provider the platform will depend on, a proposed production architecture, and a phased implementation plan. No application code has been written yet, and that is deliberate. The specification's implementation workflow (Section 61) requires an explicit approval gate between the planning phase and the build phase, and this report exists to give that gate real substance rather than a rubber stamp. Every claim in this document is labeled with an honest status: research findings are marked **VERIFIED** where they come from current official documentation, and every limitation, cost, and external dependency is stated plainly rather than smoothed over.

The first and most important audit finding is that **there is no existing project to transform**. The workspace contains a git repository with a single initial commit, a placeholder environment file pointing at a SQLite URL, and no source code of any kind. The specification was written as though it would be applied to an existing codebase, but the actual starting point is a greenfield build. This changes the reading of the audit phase: there is no legacy functionality to preserve, no migration risk to manage, and no partially-working subsystem to rescue. The audit effort therefore redirected toward verifying the external world instead, because that is where the real risks live.

Thirteen external providers and constraints were researched against current official sources (September 2026). The three findings with the largest architectural impact are: first, **MiniMax H3 is a real, currently-shipping open-weight multimodal video model** with a documented API, but its self-hosting license prohibits commercial use without written permission from MiniMax, which means production use must route through the official API; second, **WhatsApp, Instagram, Telegram and TikTok on mobile cannot accept a third-party virtual camera**, because mobile operating systems only expose physical camera devices to those apps, which confirms that the phone-to-phone physical adapter described in the specification is the only honest integration path; third, the free tier of Vercel's Hobby plan is restricted to personal, non-commercial use, so a commercial launch requires either the \$20/month Pro plan or self-hosting.

0 lines

of platform code exist today - this report is the gate before any is written

The proposed architecture follows the specification's three-plane model exactly: a control plane (Next.js, Better Auth, PostgreSQL with Drizzle, the AI Brain, jobs, consent, and administration), a media plane (LiveKit WebRTC SFU for all real-time camera and microphone transport), and a compute plane (provider-independent GPU workers speaking a registered protocol, with RunPod as the initial host and MiniMax H3 accessed as a managed API capability). The development stack runs on \$0 upfront using free tiers; the report includes an honest table of the costs that cannot be avoided at commercial launch, so the \$0 claim is never overstated.

The implementation plan proposes six phases with a vertical-slice strategy: one thin end-to-end path (sign-up through live session with a real worker and visible output) is proven first, then each plane is widened. The plan, the risks, and the open decisions that require your approval are collected in the final chapter. The build does not begin until that approval is given.

Workspace inventory

The audit inspected the repository structure, package configuration, environment variables, and all source directories. The complete inventory is short enough to list in full, and it is presented below without embellishment. A single initial commit exists, authored at workspace creation time, containing only the gitignore and an environment stub. There is no frontend, no backend, no API routes, no database schema, no authentication, no worker implementation, no mobile project, no WebRTC layer, and no admin functionality of any kind.

Area	Present state	Classification
Repository	Git repo, 1 initial commit (.gitignore only)	Empty scaffold
Environment file	DATABASE_URL placeholder pointing to local SQLite file	Stub, will be replaced
Frontend / backend	No package.json, no source files, no routes	Not implemented
Database / ORM	No schema, no migrations, no DB server	Not implemented
Auth / RBAC	Nothing exists	Not implemented
Workers / AI / media	No worker code, no WebRTC, no model integration	Not implemented
Admin / analytics / legal	Nothing exists	Not implemented
Tests / CI / deployment	Nothing exists	Not implemented

Table 2.1 - Complete audit inventory of the current workspace

Audit conclusions against the nine required classifications

The specification asks the audit to classify what works, what partially works, what is broken, what is simulated, and what is insecure. Applying those categories honestly to an empty repository: nothing works, nothing partially works, nothing is broken, nothing is simulated, and nothing is insecure, for the simple reason that nothing exists. What should be kept is nothing; what should be replaced is nothing; what should be refactored is nothing; and what is missing is everything. The practical consequence is positive: the build starts from a clean slate with no technical debt, no data migration burden, and full freedom to implement the correct architecture from day one. The only existing artifact, the SQLite environment variable, will be discarded in favor of a managed PostgreSQL connection string; local SQLite remains available for offline unit tests if useful.

The audit also examined the build specification itself for internal risks that should be surfaced now rather than discovered during implementation. Four material constraints emerged and are carried through the rest of this report: the mobile virtual-camera impossibility described in Chapter 3; the H3 licensing constraint that forces an API-first model strategy; the Nigeria Data Protection Act obligations that apply to face and voice data; and the fact that app-store distribution, custom domains, and payment collection each require external accounts and spending that no engineering effort can substitute for. None of these are reasons to abandon the project; all of them are reasons to plan around them explicitly, which the proposed architecture does.

3. Verified Provider Research

Research was conducted in September 2026 using web search against provider documentation, pricing pages, and official terms. The table below records the verified state of each provider the specification names. Prices and limits change; every figure here should be re-verified at contracting time, and the build plan includes a pre-launch re-verification step. Where a claim could not be verified from an authoritative source, it is marked accordingly rather than asserted.

Provider	Verified status and free tier	Commercial constraint
Better Auth	VERIFIED: actively maintained; took over Auth.js stewardship Sep 2025. Free, open source, TypeScript-first.	None; standard OSS license
Cloudflare R2	VERIFIED: 10 GB storage/month free, 1M Class A + 10M Class B ops free, zero egress fees; then \$0.015/GB-month.	Pay-as-you-go beyond free tier
Cloudflare Realtime	VERIFIED: SFU + TURN; TURN free when used with the SFU; \$0.05/GB outbound after 1 TB free.	Newer product; smaller SDK ecosystem than LiveKit
LiveKit	VERIFIED: open source, self-host free; Cloud free tier includes 1,000 agent-session minutes and 40,000 transport requests monthly.	Cloud paid tiers meter per minute
Neon Postgres	VERIFIED: free plan 0.5 GB storage per project, 100 compute-hours/month, scale-to-zero.	Storage ceiling for free tier
Vercel	VERIFIED: Hobby plan \$0 (100 GB bandwidth, 1M invocations) but is for personal, non-commercial use.	Commercial launch requires Pro at \$20/month
Railway	VERIFIED: one-time \$5 trial credit (30 days); thereafter \$1/month free credit, Hobby \$5/month + usage.	Not free for production hosting
RunPod	VERIFIED: RTX A5000 community pods from roughly \$0.16-0.27/hour; serverless tier from about \$0.58/hour.	Usage-based; region-dependent price/availability
Kaggle	VERIFIED: 30 GPU-hours/week per accelerator type, quota resets Saturday.	Terms restrict use to non-commercial; unsuitable as production backbone

Provider	Verified status and free tier	Commercial constraint
MiniMax H3	VERIFIED: shipping open-weight multimodal model (Aug 2026) with official API; accepts up to 9 images + 3 videos + 3 audio inputs for consistency.	Self-hosted commercial use requires written permission from MiniMax; production must use official API. Reported self-host geo-restrictions require license review
PostHog	VERIFIED: free tier 1M analytics events, 5K session recordings, 1M feature-flag requests monthly.	Usage-based beyond
NDPA (Nigeria)	VERIFIED: Nigeria Data Protection Act 2023; NDPC regulator; biometric data is sensitive; high-risk processing triggers DPIA duties.	Registration duties for controllers of major importance; legal counsel required

Table 3.1 - Verified provider status (researched September 2026)

Critical finding 1 - MiniMax H3 is real, and its license decides the architecture

The specification instructs not to guess whether H3 exists or what it can do. Verified current state: MiniMax H3 and H3-Max are a new-generation general-purpose multimodal video generation system, published as open weights on Hugging Face (August 2026) and offered through the official MiniMax platform API and third-party inference hosts. The model accepts multiple reference images, videos and audio tracks in a single request specifically to preserve character and voice consistency, which aligns directly with this platform's character-fidelity requirement. Community reporting (marked **REQUIRES VERIFICATION** against the current **LICENSE** file before any commitment) indicates the self-hosting license prohibits commercial use without prior written permission, and that self-hosted use may carry geographic restrictions. The architectural consequence adopted in this report: **H3 is treated as a capability delivered through the official API for all production traffic, and self-hosting is confined to development and evaluation only**. This converts an uncertain licensing problem into a metered, legitimate operating cost, and it is the reason the compute plane exposes models as swappable capabilities rather than hard-coded dependencies.

Critical finding 2 - mobile virtual cameras are impossible; the phone-to-phone adapter is the honest path

Verified against WhatsApp's own documentation and multiple platform sources: mobile apps such as WhatsApp, Instagram, Telegram, and TikTok enumerate physical camera devices and provide no mechanism for a third-party application to register a virtual camera on iOS or Android. On desktop, tools like OBS can inject a virtual camera into some apps; on mobile, the operating system simply does not offer this to sandboxed third-party apps. The specification anticipated exactly this risk and mandated the fallback: a phone-to-phone physical camera adapter, where the first phone displays the AI-transformed live output fullscreen while a second phone physically films that screen for use inside any social app. This report therefore treats phone-to-phone as a first-class output adapter, honestly named a physical-camera adapter, and the product copy will never claim direct integration with WhatsApp, Instagram, TikTok, Telegram, or Facebook unless such integration is actually built and verified on target devices.

Critical finding 3 - hosting terms shape the launch decision

Vercel's Hobby plan is free but explicitly limited to personal, non-commercial projects. For development and internal demos this is perfectly appropriate; the moment the platform serves real users as a business, the correct choices are Vercel Pro (\$20/month) or a self-hosted alternative such as a small VPS with Coolify or a Railway Hobby workspace. This report recommends deciding at Phase 5 (launch preparation) rather than now, because the control plane is a standard Next.js application and remains portable across all three options. The same honesty applies to app stores: distributing the mobile application requires an Apple Developer account (\$99/year) and a Google Play account (\$25 one-time), both **REQUIRES EXTERNAL CREDENTIAL** items that no code can substitute.

4. Proposed Three-Plane Architecture

The architecture adopts the specification's three-plane separation without modification, because the separation itself is the primary defense against the classic failure mode of media platforms: choking ordinary request/response infrastructure with high-frequency video traffic. Each plane has a distinct responsibility, a distinct failure domain, and a distinct scaling profile. The control plane holds state and truth; the media plane moves real-time streams; the compute plane does GPU work. The diagram below shows the arrangement, and the sections after it describe each plane in turn.



Figure 4.1 - Three-plane architecture with transport boundaries

Control plane

The control plane is the system of record. It owns authentication and sessions (Better Auth), authorization and role checks, users and organizations, characters and their face/voice assets, projects, live-session state machines, job records, credits and billing state, consent records, moderation queues, audit logs, feature flags, system configuration, the provider and worker registries, the capability registry, and the AI Brain with its tool registry. It is implemented as a Next.js application with server-side validation on every route, backed by PostgreSQL through Drizzle ORM. Nothing in the control plane ever touches raw video frames; it issues commands, records facts, and observes status. This keeps its scaling profile ordinary (stateless API plus a relational database) even as the media and compute planes scale in far more dramatic ways.

Media plane

The media plane is LiveKit. All camera and microphone transport travels over WebRTC: the client publishes its tracks to a LiveKit room, and GPU workers join that same room as participants to receive the inbound track and publish the transformed outbound track back to the client. This is the worker-as-participant pattern LiveKit was designed around, and it guarantees the specification's rule that raw live video never passes through ordinary REST endpoints. LiveKit is chosen over building raw WebRTC and over Cloudflare Realtime for the initial build because its agent framework and client SDKs directly implement this pattern; Cloudflare Realtime remains a documented fallback with attractive pricing (\$0.05/GB after the first TB free) should LiveKit costs or operations become a problem. The media plane also owns recordings: completed session recordings are written to object storage, never to the database itself.

Compute plane

The compute plane is a fleet of authenticated GPU workers speaking a provider-independent protocol (Chapter 8). Workers run on RunPod pods (pay-as-you-go, scaled to zero when idle) and execute face processing, voice processing, avatar generation, lip synchronization, and video generation tasks. The MiniMax H3 capability is reached through the official API as a managed provider rather than a self-hosted model, per the licensing constraint in Chapter 3. FFmpeg runs inside workers under a strict execution policy: argument allowlists, validated file paths, resource limits, and timeouts, with uploaded media always treated as untrusted input. Kaggle may be used for development experiments within its 30-hour weekly quota but is never a production worker, per its terms. The plane exposes exactly one thing to the control plane: a worker identity, its capabilities, and its health.

Why the separation matters

Three concrete reasons. First, failure isolation: a GPU driver crash in the compute plane must never take down login, billing, or session records, and a LiveKit outage must not corrupt job state. Second, independent scaling: on a launch day, media bandwidth and GPU capacity are the scarce resources, and the three-plane layout lets each be scaled (and billed) on its own axis without over-provisioning the others. Third, provider independence: because workers register through a protocol rather than being hard-wired to RunPod or any GPU vendor, and because models are entries in a capability registry rather than names burned into business logic, replacing any single provider is a configuration change, not a rewrite. This is the property that makes the platform honest about its own infrastructure instead of married to it.

5. Technology Decisions and Rationale

Every technology below was selected against the specification's criteria: reliability, cost, free-tier availability, accessibility from Nigeria, performance, security, scalability, operational complexity, mobile compatibility, WebRTC compatibility, GPU compatibility, and commercial licensing. Alternatives considered and rejected are listed with reasons, because a decision without a rejected alternative is a decision that has not really been made.

Layer	Decision	Why	Rejected alternative
Web app	Next.js + TypeScript	Server-side validation, API routes colocated with UI, mature ecosystem, deployable on Vercel/VPS alike; strict TS catches contract drift early.	Plain SPA + separate API (more moving parts, no SSR benefit here)
Auth	Better Auth	Actively maintained, took over Auth.js stewardship Sep 2025; email/password, OAuth, passkeys, 2FA, orgs as plugins; secure HTTP-only cookies; free.	Clerk (per-MAU cost), custom crypto (unjustifiable risk)
Database	PostgreSQL (Neon) + Drizzle ORM	Relational fits users/characters/jobs/consent/audit; Drizzle is typed, migration-first, no runtime magic; Neon free tier for dev, scale later.	MongoDB (schemaless harms integrity of consent/audit records)
Job queue	pgboss on Postgres	Durable, transactional with job records, zero extra infrastructure; sufficient at launch scale.	Redis + BullMQ now (extra service to secure/operate before it is needed)
Realtime media	LiveKit	Worker-as-participant agent pattern, mature JS/RN SDKs, OSS self-host option, Cloud free tier for dev.	Cloudflare Realtime (cheaper per GB but younger agent story - kept as fallback); raw WebRTC P2P (no SFU, no multi-worker routing)
Object storage	Cloudflare R2	10 GB free, zero egress (critical for video), S3-compatible SDK, presigned URLs, lifecycle rules.	S3 (egress fees), Vercel Blob (smaller free tier)

Layer	Decision	Why	Rejected alternative
GPU workers	RunPod pods, provider-independent protocol	Pay-per-second pods, scale to zero, container-friendly; protocol keeps vendor swappable.	Reserved GPU servers (fixed cost), Kaggle (non-commercial terms)
Video model	MiniMax H3 via official API	Character/voice consistency inputs; open weights exist but self-host commercial use needs written permission - API route is legitimate and metered.	Self-hosting H3 (license risk), unverified closed providers
Analytics	PostHog, consent-gated	1M events/month free covers launch; feature flags included; session recording disabled by default.	Self-hosted analytics (operational cost without benefit at this scale)
Mobile	Expo React Native	True native app (not a WebView); camera/mic permissions, keep-awake fullscreen output, session recovery; over-the-air updates.	WebView wrapper (explicitly forbidden by spec), native Swift/Kotlin (2x codepaths)

Table 5.1 - Technology decisions with rationale and rejected alternatives

Two decisions deserve explicit honesty about their boundaries. The pgboss-over-Redis choice is a launch-scale decision, not a permanent one: if worker heartbeat volume or queue depth ever makes Postgres the bottleneck, Redis enters the architecture at that point, behind the same queue interface, and nothing above that interface changes. Similarly, LiveKit self-hosting on a modest VPS is the \$0-friendly path and LiveKit Cloud with its free tier is the low-operations path; the report recommends starting on whichever the operator prefers and treats switching as configuration rather than surgery, because the client code speaks LiveKit's protocol identically in both cases. The unifying principle across all decisions is the specification's own: prefer simple systems that actually work over impressive systems that might.

6. Cost and Free-Tier Analysis

The specification's cost philosophy is adopted verbatim: \$0 upfront, then free tiers, then pay-as-you-go, then user-funded usage, and paid infrastructure only when justified. The table below shows the honest shape of that ladder for this platform. Development and internal testing genuinely run on \$0 across every component. The second table lists the costs that cannot be engineered away; they are external accounts, fees, and usage-based charges that arrive with commercial operation. Presenting both tables together is deliberate: the platform can truthfully say it costs nothing to build and test, while never claiming it costs nothing to run.

Component	Provider / tier	What the free tier covers
Web + API hosting	Vercel Hobby	100 GB bandwidth, 1M function invocations - fine for development and internal demos (non-commercial use only)
Database	Neon Free	0.5 GB storage, 100 compute-hours/month, 10 branches - sufficient for schema development and tests
Object storage	Cloudflare R2	10 GB storage, 1M Class A + 10M Class B operations, zero egress

Component	Provider / tier	What the free tier covers
Realtime media	LiveKit OSS self-host or Cloud free tier	Self-host on a local machine for development; Cloud free tier provides 1,000 agent minutes + 40K requests/month
Auth	Better Auth	Open source, no metering
Analytics	PostHog Free	1M events, 5K recordings, 1M flag requests monthly
GPU compute (dev)	Kaggle	30 GPU-hours/week for development experiments within non-commercial terms
Job queue	pgboss	Runs inside the existing Postgres - no additional service

Table 6.1 - \$0 development stack (all verified free tiers)

Item	Cost (verified)	When it is needed
Vercel Pro (or VPS/Railway)	\$20/month (Pro)	At commercial launch - Hobby tier is non-commercial
Apple Developer account	\$99/year	Before shipping the iOS app
Google Play account	\$25 one-time	Before shipping the Android app
Custom domain	~\$10-15/year	Launch; also enables HTTPS certificates
MiniMax H3 API usage	Pay-per-request; promo pricing observed on some hosts	Every production generation job
RunPod GPU time	~\$0.16-0.27/hr per A5000 community pod	Live/face/voice sessions and batch jobs; scales with usage
LiveKit at scale	Self-host VPS (~\$5-20/month) or Cloud paid tier	When free tiers are exceeded by real traffic
Postgres at scale	Neon paid tier when >0.5 GB / compute limits	With user growth
Payment provider	Not yet selected - requires verification for Nigeria	Before charging users; until then credits are manually granted

Table 6.2 - Costs that arrive with commercial operation (cannot be engineered away)

The platform's user-funded usage model is the credit system: generation and live-session minutes are metered per job and priced in credits, so GPU and API costs track revenue instead of preceding it. Cost control on the infrastructure side comes from the compute sleep system (Chapter 8): workers scale to zero when idle, so the platform pays for GPU capacity only while sessions or jobs actually run. The single largest variable cost is live-session GPU time; at the observed RunPod community rate of roughly \$0.27 per A5000 hour, a ten-minute live session costs on the order of \$0.05 of raw compute before model API costs, which sets a sane ceiling for credit pricing. Payment collection in Nigeria requires a provider decision (Paystack, Flutterwave, or similar) that is explicitly out of Phase-1 scope and flagged in the approval gate; until a provider is integrated and verified, no charging occurs and refunds are handled as manual credit adjustments, which is exactly what the refund policy will state.

7. AI Brain Architecture

The Brain is the platform's orchestrator: the component that turns a user or administrator intent into a validated, authorized, policy-checked execution against registered tools. Its architecture is a pipeline, and each stage exists to make the next stage's job impossible to skip. An LLM never executes anything itself; it only proposes a structured tool request. Everything after that proposal is deterministic code.

The pipeline, in order: the LLM receives the user intent plus a manifest of available capabilities, and emits a structured tool request as JSON. Schema validation rejects any request that does not match the tool's registered input schema, before any authorization runs. The authorization stage resolves the requesting user's roles and permissions and checks them against the tool's required permissions - this is a database check, not something the LLM is asked politely to respect. The policy engine then applies platform rules that can veto regardless of permissions: safety policy, consent requirements for the referenced assets, plan limits, rate limits, and feature flags. Only then does execution dispatch the tool's handler, whose result is recorded on the job and written to the audit log with actor, tool, inputs, decision, and outcome. Failures at any stage are returned as structured errors, never as free-form text that the caller must guess about.

Tool attribute	Purpose	Enforced by
Name + description	Unique identity; what the tool does	Registry rejects duplicates; LLM sees only descriptions
Input schema	Strict JSON schema for arguments	Validation stage rejects non-conforming requests pre-authorization
Required permissions	Granular permission strings	Authorization stage (server-side DB lookup)
Rate limits	Per-user and per-plan ceilings	Policy engine
Consent requirements	Asset/purpose scopes the tool consumes	Policy engine cross-checks consent records
Failure handling	Retryable vs terminal; idempotency key	Job system
Audit logging	Actor, inputs, decision, result	Audit append at every execution and every veto

Table 7.1 - Brain tool registry contract (every tool, no exceptions)

What the Brain can never do

The Brain has no shell, no filesystem scope beyond its job inputs, no database credentials, and no ability to mutate its own permission set. It cannot execute arbitrary shell commands; it cannot modify security controls; it cannot grant itself permissions; it cannot read arbitrary secrets (secrets are held by the platform and injected only into the specific tools that need them); it cannot bypass authorization, billing, or consent checks; it cannot delete production data outside explicitly registered, narrowly scoped tools; and it cannot disable audit logging. These are not instructions to the model - they are properties of the runtime: the only actions that exist are the registered tools, and every tool passes the same pipeline. Prompt-injection defense follows the same structure: untrusted content (user text, uploaded filenames, media metadata) is framed as data, never as instructions; the tool manifest is the only source of available actions; and validation plus the policy engine run regardless of what the model proposes. An injected instruction can at worst produce a proposal that the pipeline then rejects.

Model independence

The Brain's LLM is a swappable provider behind an interface, because the specification's warning about guessing model capabilities applies to the orchestrator as much as to video models. The initial implementation will use a verified, currently-available model with proven structured-output support, configured via environment variables, with the ability to change provider without touching pipeline code. The Brain's intelligence is deliberately distributed: the LLM contributes intent parsing and tool selection, while worker selection, cost, latency, and health routing live in deterministic code in the scheduler, where they can be tested exhaustively (Chapter 14) rather than probabilistically.

8. GPU Worker System

Workers are the platform's muscle, and their design principle is provider independence: the control plane never knows whether a worker is a RunPod pod, a future GPU host, or a development machine - it knows a worker identity, its capabilities, and its health, all expressed through one protocol. This is what makes Kaggle's limitations (non-commercial terms, 30-hour weekly quota) a non-event: Kaggle is simply one dev-tier worker, and its eventual irrelevance costs nothing.

Operation	Purpose	Auth
REGISTER	Worker announces provider, GPU, VRAM, models, capabilities, version, region	Worker credential (issued by admin, rotatable)
HEARTBEAT	Liveness + current load, active jobs, latency; missed heartbeats trigger failure handling	Signed with worker credential
HEALTH	On-demand deep check: memory, model availability, disk, errors	Worker credential
CAPABILITIES	Full capability + model manifest refresh	Worker credential

Operation	Purpose	Auth
RESERVE	Control plane holds a worker for an incoming job	Control-plane initiated
ALLOCATE	Job bound to worker; idempotency key attached	Control-plane initiated
LOAD_MODEL	Load the required model into memory	Worker validates against its own manifest
START / STOP	Begin or cease a job execution	Control-plane initiated, audited
STREAM	Worker joins the LiveKit room as participant; media flows in the media plane	Room token scoped to session
STATUS	Per-job progress reported as real state, never fabricated	Worker credential
RELEASE	Job finished; worker returns to idle	Control-plane initiated
DRAIN	Stop assigning new work; finish current jobs; then sleep or shut down	Control-plane or admin initiated
SHUTDOWN	Clean termination	Worker credential
ERROR / RESULT	Structured failure or completion payloads for the job record	Worker credential

Table 8.1 - Worker protocol operations

Lifecycle, selection, and the sleep system

The worker lifecycle is an explicit state machine: **BOOT**, **REGISTER**, **HEALTH CHECK**, **IDLE**, **RESERVED**, **LOADING**, **READY**, **BUSY**, **DRAINING**, back to **IDLE**, and finally **SLEEP** or **SHUTDOWN**. The scheduler selects workers by the specification's full factor list - capability match, GPU type, VRAM, model compatibility, health, load, latency, region, user plan, queue depth, and cost where the provider reports it - and it enforces the spec's example rules as hard constraints: a live face-processing task is never sent to a worker that only supports offline video generation, and an H3-generation job is only ever routed to the H3 API adapter. Selection is deterministic, unit-tested code, which is what makes these guarantees trustworthy rather than aspirational.

Cost control comes from the sleep system: a worker with no job idles for a timeout window, drains, and sleeps (on RunPod, the pod scales to zero, so idle time costs nothing). When a new job arrives, the sequence is capability check, worker wake, health check, model load, ready, execute. The platform does not pretend this is instant: cold-start wake plus model load is a real delay on the order of tens of seconds to minutes depending on model size, and the session UI presents that honestly as a loading state with real backend state behind it (Chapter 9), never a fabricated progress bar.

Failure recovery

When a worker stops responding, the control plane executes a fixed recovery procedure: detect the missed heartbeats (threshold-based); mark the worker unhealthy and stop assigning new work; determine whether its in-flight jobs are recoverable (idempotent jobs may retry, streaming sessions may hand off); retry or reassign where safe; record the incident on the job and worker records; notify the session manager so the user sees a recovering state rather than a frozen one; and restore the session where technically possible. The system never pretends a dead worker is alive - health is observed state, not hope. Idempotency keys make retries safe: a re-executed job either resumes its unit of work or is recognized as already completed, and the job record's retry count and failure information are surfaced to admins.

Session state machine

Every live session is a record with an explicit state, and the user interface reflects that state in real time - the status text a user sees (connecting, finding worker, worker assigned, loading model, live, network degraded, recovering) is read from the backend session record, never simulated client-side. The states are: CREATED, VALIDATING, WAITING_FOR_WORKER, WORKER_ASSIGNED, LOADING, READY, LIVE, DEGRADED, RECOVERING, STOPPING, COMPLETED, FAILED, CANCELLED, and EXPIRED. Transitions are server-authoritative; the client may request but never set state. WebSocket events push transitions to connected clients, and a reconnecting client re-syncs state from the record rather than guessing.

Job system

Every asynchronous task - character renders, avatar generations, batch video jobs, media transcodes - has a durable job record with the fields the specification requires: job ID, user ID, project/session linkage, type, status, priority, input references, worker ID, provider, created/started/completed times, failure information, retry count, result, usage, and cost where available. Jobs are processed through the pgboss queue with idempotency keys so retries never duplicate work or billing. The status surfaced to users is the job's actual state; where a job is queued behind a sleeping worker, the UI says exactly that, with the cold-start explanation from Chapter 8.

Media storage

The storage rule is strict: the database stores metadata; objects live in Cloudflare R2. Uploads use presigned URLs with MIME and extension validation and file-size limits enforced before and during upload; downloads use short-lived presigned URLs with ownership verification on every request, so no permanent public media URLs exist by default. Every access to an asset verifies that the requesting user owns or is authorized for the object, closing the IDOR/BOLA path. R2 lifecycle rules implement retention: raw session recordings are retained per policy then expire automatically, and user-initiated deletion removes both the object and its metadata. CORS is scoped to the platform's own origins.

Output adapters

Adapter	What it actually is	Honest claim
NATIVE_WEB	Live/recorded output in the platform's own web UI	Fully supported
NATIVE_MOBILE	Live/recorded output inside the platform's own mobile app	Fully supported
WEBRTC_LIVE	Real-time transformed stream to any WebRTC-capable endpoint	Supported where the endpoint accepts a connection
PHONE_TO_PHONE	Physical-camera adapter: phone 1 shows transformed output fullscreen; phone 2 films it for use in any social app	Works with any app that uses the phone's camera - no app-specific integration claimed
OBS	Worker publishes a stream the user pulls into OBS as a media source on desktop	Desktop-only; documented setup
RECORDING	Session recorded to R2, delivered via short-lived link	Fully supported
RTMP	Future: push output to an RTMP ingest	Not implemented; declared as future capability only

Table 9.1 - Output adapter registry (capability-declared, not assumed)

The phone-to-phone fullscreen experience is engineered as the specification demands: unnecessary UI hidden, screen kept awake, aspect ratio preserved, portrait orientation supported, optional mirror mode, stable framing, minimal status information, a low-bandwidth mode, and automatic recovery from connection loss. Its documentation and marketing copy will describe it accurately as a physical-camera adapter and will not claim that WhatsApp, Telegram, Instagram, TikTok, or Facebook accept a virtual AI camera, because on mobile they do not (Chapter 3, verified).

Web application

The web application is a single Next.js codebase containing the marketing site, the product application, and the admin console, with server-side authorization on every route. User-facing routes: landing page, sign in/up, dashboard, projects, character studio (create and manage characters: face, appearance, voice, personality configuration, style, language, consent status, source assets, ownership), live studio (start, monitor, and end live sessions with real state), media library, recordings, settings, account management, consent management, credits (no payment provider until one is verified and integrated), in-app help and documentation, and the legal pages (terms, privacy, cookies, refunds, acceptable use, copyright, accessibility, voice rights, abuse). Admin routes are behind the RBAC wall described in Chapter 11. The design follows the specification's anti-slop rules: no purple-gradient aesthetic, no glassmorphism excess, no fake counters or testimonials, no decorative robots or holograms, no emoji icons, and motion that communicates rather than decorates. The hero states what the product does and shows real functionality.

Mobile application

The mobile app is a true Expo React Native application, not a WebView wrapper. It supports camera and microphone permissions with proper platform flows, live sessions with transformed preview, character selection, media upload and download, session recovery, network recovery, low-bandwidth operation (lower resolution, bitrate, and frame rate, with an audio-first fallback), orientation handling, fullscreen live output with keep-awake, account management, and notifications where implemented. Platform-specific APIs (keep-awake, permissions, file system) are used where required. Performance budgets from the specification are treated as engineering requirements: resolution, bitrate, frame rate, memory, battery, CPU/GPU usage, image sizes, and background behavior are tuned, and degraded modes exist for constrained devices. Distribution requires the external developer accounts and app-store review listed in Chapter 6; the plan phases mobile after the web platform is stable, which is the honest sequencing for a solo-to-small engineering effort.

3D usage policy

3D is used only where it genuinely improves the product: character visualization and live preview, camera framing, and studio context. Desktop gets richer assets; mobile gets optimized ones; every 3D surface has a non-3D fallback; and reduced-motion preferences disable non-essential animation throughout. Nothing on the marketing or product site exists purely to look futuristic - if a visual does not communicate something real about the platform, it does not ship.

11. Security Model

Security decisions follow the specification's priority order: when cheap and secure conflict, secure wins, then a cheaper secure implementation is found. The model below is designed to be implemented and tested as part of the build phases, not bolted on afterward. Frontend authorization is never trusted: every route validates server-side, and every object access verifies ownership.

Threat	Control
Broken authentication / session hijacking	Better Auth with secure HTTP-only cookies, trusted-origin configuration, session revocation, 2FA and passkeys available
IDOR / BOLA (object access)	Ownership verification on every asset, job, session, character, and presigned URL; no enumerable IDs without authorization
Privilege escalation	Server-side RBAC with granular permissions; roles never read from client input; admin actions all audited
SQL injection	Drizzle parameterized queries everywhere; no string-built SQL
XSS / CSRF	Framework output encoding; CSRF-safe cookie strategy and origin checks on state-changing routes
SSRF	Outbound fetch allowlists; no user-supplied URLs fetched server-side

Threat	Control
Malicious uploads / FFmpeg abuse	MIME + extension + size validation, magic-byte checks, quarantined processing directory, argument allowlists (user text never reaches argv), CPU/time/memory limits, no shell interpolation
Worker spoofing / impersonation	Workers authenticate with rotatable credentials; capability claims validated at health checks; heartbeats signed; revocation revokes access immediately
Brain tool abuse / prompt injection	Registered tools only; schema validation before authorization; untrusted content framed as data; policy engine veto; full audit trail
Rate-limit bypass / API abuse	Per-user, per-plan, per-IP limits at the edge and in the policy engine; abuse monitoring in admin
Secret leakage	Server-only env vars, .env.example documentation, secrets never in client bundles, admin UI shows redacted references only
Replay / duplicate jobs	Idempotency keys on job creation and worker execution
CORS misconfiguration	Scoped origins; no wildcard on credentialed routes

Table 11.1 - Threat model and controls

Roles and administration

Roles: USER, MODERATOR, SUPPORT, ADMIN, SUPER_ADMIN, with granular permissions underneath (for example, 'workers:drain' separate from 'users:ban'), because the specification is explicit that not every admin should have every permission. Administrative access requires MFA. The admin console is a first-class part of the control plane, not a hidden backdoor: its routes are behind the same RBAC and audit machinery, and every emergency control (disable capability, disable provider, stop new jobs, maintenance mode, disable uploads, disable live sessions, emergency moderation) writes an audit entry naming the actor, the action, and the reason prompt. The admin UI never displays raw secrets - only redacted references - and the worker registry exposes exactly the operational data the specification lists (provider, GPU, VRAM, status, health, active jobs, queue, model, capability, heartbeat, errors, latency).

Nigeria Data Protection Act posture

The platform will be available in Nigeria, so the NDPA 2023 and NDPC guidance are treated as design inputs, not afterthoughts. Verified legal facts that shape the design: face templates and voice prints are biometric data, which the NDPA classifies as sensitive personal data; processing sensitive data at scale is high-risk processing, which engages data-protection impact assessment duties; data controllers of major importance (including entities processing high volumes of data subjects or high-risk categories) have registration duties with the NDPC; data subjects hold rights of access, correction, and deletion; and cross-border processing (this platform's compute runs on US/EU-hosted GPU and API providers) must be handled consistently with the Act's transfer requirements. This report is an engineering document, not legal advice: **professional legal review is required before launch**, and the approval gate lists it as a blocking external item. The engineering commitments that make such review tractable are below.

Privacy data map and retention

Data type	Purpose	Stored where	Retention / deletion
Account data	Identity, authentication	Postgres	Deleted on account deletion request; audit stubs retained per law
Face assets / templates	Character transformation	R2 (encrypted at rest) + Postgres metadata	Deleted on withdrawal or account deletion; consent-scoped
Voice assets / prints	Voice transformation	R2 + Postgres metadata	Same as face assets
Session recordings	User's own output history	R2	Lifecycle expiry (default 30 days, configurable); immediate on user deletion
Job records / usage	Billing, support, audit	Postgres	Anonymized after retention window
Audit logs	Security and abuse evidence	Postgres (append-only)	Retained per security policy; not erased on account deletion (legal basis: legitimate interest / legal obligation)
Analytics events	Product analytics	PostHog, consent-gated	Deleted on consent withdrawal; session recording off by default

Table 12.1 - Privacy data map (engineering commitments; full map ships with the platform)

Consent system

Face and voice transformation requires explicit, purpose-scoped consent records - not a generic terms checkbox. Each record stores the user, the asset, the purpose, the scope, the timestamp, the policy version, the authorization status, and the withdrawal status, and withdrawal is a first-class action that disables further processing of that asset, enforced by the policy engine in the Brain pipeline. The consent flow also covers the platform's own acceptable-use boundary: creating characters of real people requires authorization from that person, and the abuse-report path (impersonation, unauthorized likeness or voice, non-consensual sexual content, harassment, fraud, copyright) is built into the product surface, with moderator queues and documented decisions in the admin console.

Cookies, legal pages, and honest claims

Tracking is classified (NECESSARY / PREFERENCES / ANALYTICS / MARKETING) with non-essential tracking off until consent; the cookie control offers Accept All, Reject Non-Essential, and Manage Preferences, and withdrawal is always available. Legal pages ship at the routes the specification lists (terms, privacy, cookies, refunds, acceptable use, copyright, accessibility, voice rights, abuse) with real operator information, effective dates, and contact routes - no invented entities or registration numbers, and no copied boilerplate. The refund policy will state exactly what the billing system (manual credits, pre-payment integration) can actually do, per the specification's rule that refund promises must match billing reality. Product copy across the site is written against the prohibited-claims list: no fastest/best/instant/zero-latency/unlimited/100%-private/bank-grade/guaranteed language, and no performance or security claims that have not been measured.

13. Accessibility Plan (WCAG 2.2 AA)

The platform targets WCAG 2.2 AA, and the plan treats accessibility as a build-phase requirement with a test gate, not a post-launch cleanup. The commitments: color contrast at or above AA ratios, with meaning never communicated by color alone; complete keyboard navigation with visible, well-ordered focus indicators across every flow including the live studio and dialogs; semantic HTML with correct heading structure, real labels on every form control, and accessible error messaging that is announced, not merely rendered; dialogs and menus that trap focus appropriately and close predictably; alt text for every meaningful image and a text alternative strategy for live-status changes (live regions announcing session state transitions); responsive reflow at 400% zoom without horizontal scrolling; touch targets at comfortable sizes for mobile; prefers-reduced-motion honored across all animation, including the 3D system's non-essential motion; and loading, empty, and error states that are programmatically communicable, not just visible.

Verification is tooling plus human passes: automated checks (axe-core) run in CI on every page state; manual keyboard walkthroughs cover the critical journeys (sign up, create character, grant consent, start live session, report abuse); screen-reader smoke tests (VoiceOver on the web/mobile builds, NVDA on Windows) run before each release; and any new icon-only control requires a documented accessible name before merge. The accessibility statement page will disclose the target level and the reporting channel for problems, and accessibility bugs are triaged with the same severity discipline as security bugs - a blocking regression in a critical flow stops a release.

14. Testing Plan

Testing is layered to match the architecture: unit tests lock down deterministic logic, integration tests exercise real infrastructure boundaries, end-to-end tests walk the user's actual journey, and dedicated failure and security suites attack the system on purpose. The specification's testing matrix is adopted in full; the table below maps it to concrete suites that become exit criteria for the phases in Chapter 16.

Suite	Covers	Exit criterion
Unit	Brain tool schemas, validation, authorization logic, worker selection (capability/VRAM/plan routing), session and job state machines, consent enforcement, permission checks, storage access guards	All pass in CI on every commit; no skipped tests
Integration	Auth flows against real Postgres, storage presign/verify, worker REGISTER/HEARTBEAT/ALLOCATE lifecycle, job execution through pgboss, session creation, media processing through FFmpeg sandbox, admin actions	All pass against a fresh environment per run
End-to-end	Signup, login, create character, upload asset, grant consent, start session, worker assignment, processing, live output, stop, save result	Green on the vertical-slice environment (Phase 1 exit)
Failure	Worker crash mid-job, network loss mid-session, model load failure, storage failure, API timeout, duplicate job submission, expired session, revoked consent mid-flow, unauthorized access attempts, malicious upload	Each scenario produces the specified recovery/rollback behavior, not a hang or silent loss
Security	Cross-user access, user-into-admin attempts, forged roles, expired/invalid sessions, prompt injection, worker spoofing, rate-limit bypass, SSRF/XSS/CSRF probes, path traversal on filenames	Zero critical findings open at any phase exit; findings filed with severity
Accessibility	axe-core CI on key states; keyboard walkthroughs; screen-reader smoke tests	Zero serious violations on critical journeys

Table 14.1 - Test suites and coverage targets

Two honesty rules govern the test reports: a feature is only declared **TESTED** when its suite runs against real infrastructure (a unit test of the worker protocol against a mock is **PARTIALLY TESTED** until an integration run exercises a real worker), and test results are reported with the specification's status vocabulary (**IMPLEMENTED** / **TESTED** / **PARTIALLY TESTED** / **BLOCKED** / **NOT IMPLEMENTED** / **REQUIRES EXTERNAL CREDENTIAL** / **REQUIRES PROVIDER** / **REQUIRES MANUAL VERIFICATION**) rather than a bare done.

Topology and environments

The deployed system has six moving parts: the Next.js application (Vercel for development; Pro/VPS/Railway decision at launch), the PostgreSQL database (Neon), the R2 object storage bucket, the LiveKit media server (self-hosted VPS or LiveKit Cloud), GPU workers (RunPod, deployed as container images booted on demand), and the external MiniMax H3 API. Development, staging, and production are separate environments with separate credentials; migrations run through Drizzle's migration tooling in CI before promotion; and every deploy is a repeatable pipeline, not a laptop. The worker container image is versioned and its version is part of the REGISTER payload, so the control plane can refuse outdated workers.

Environment variables and secrets

Class	Examples	Exposure
PUBLIC	NEXT_PUBLIC_APP_URL, analytics opt-in flags	Client-safe by design
SERVER	AUTH_SECRET, session config, rate-limit keys	Server only; never prefixed NEXT_PUBLIC
DATABASE	DATABASE_URL (Neon), pool settings	Server only
STORAGE	R2 keys, bucket, region, presign TTLs	Server only
MEDIA	LiveKit URL + API key/secret	Server only; room tokens minted per session
WORKER	WORKER_ID, control-plane URL, worker credential	Worker containers only
AI	MiniMax API key, Brain LLM provider key	Server / worker only
ADMIN	MFA enforcement, emergency flag defaults	Server only
PAYMENT	(empty until a verified provider is integrated)	Deferred; see approval gate

Table 15.1 - Environment variable classes (documented in `.env.example`; never committed)

Operations

Observability is layered from day one: structured request and job logs, health endpoints for the app and database, worker heartbeat and health telemetry surfaced in the admin console, error tracking, and the append-only audit log. Backups: Neon's built-in point-in-time recovery covers the database; R2 objects are replicated by Cloudflare's storage durability, and the job records in Postgres are the source of truth for regenerating any derived media that is lost; configuration recovery is a solved problem because all configuration lives in versioned migrations and environment templates. Monitoring covers queue depth, worker health, session failure rates, and cost signals (RunPod hours, H3 API usage), so cost anomalies are visible in the admin console rather than on a surprise invoice. The incident runbook documents: kill a provider (capability off), drain workers, roll back a bad deploy, restore from backup, and the emergency controls, each with its audit trail.

16. Implementation Phases

The build order is a vertical slice first, then widening - the strategy that produces a working, testable system at every phase boundary instead of a long silence followed by an integration disaster. Each phase has concrete exit criteria in the specification's honest-status vocabulary, and no phase begins until the previous one is TESTED (or its deviations are explicitly logged and approved). Durations are given as focused-build ranges for a solo developer plus automation; they are estimates for planning, not promises.

Phase	Scope	Exit criteria (status vocabulary)	Est.
P0 Foundations	Repo, CI, TypeScript strict, Drizzle schema + migrations for users/sessions/jobs/consent, Better Auth (email+password, sessions, RBAC roles), .env.example, audit log table	Auth flows TESTED (integration); schema migrated on dev DB; CI green	1-2 wk
P1 Vertical slice	Characters CRUD + asset upload to R2 (presigned, validated); consent records; job system (pgboss, idempotency); one real GPU worker (dev machine) implementing the worker protocol; LiveKit room + worker-as-participant with a real (simple) transform; session state machine + WebSocket status; minimal live studio UI showing true state	Full E2E journey signup through save result TESTED against real worker; failure suite for worker-crash and network-loss TESTED	3-4 wk
P2 Compute reality	RunPod worker image + sleep system; MiniMax H3 via official API as managed capability; scheduler full-factor selection; job cost/usage recording; recordings to R2 with lifecycle rule	Job through H3 API TESTED; RunPod wake/sleep TESTED; cold-start UX reflects real state	2-3 wk
P3 Trust plane	Moderation reports + queues; admin console (users, content, workers, emergency controls, audit view); granular permissions + MFA for admin; rate limits; analytics consent gating	Admin actions audited TESTED; security suite zero critical findings	2-3 wk
P4 Mobile	Expo app: permissions, live session view, character selection, uploads/downloads, session + network recovery, fullscreen keep-awake output, low-bandwidth mode	Camera/mic TESTED on real Android + iOS devices; recovery flows TESTED	3-4 wk
P5 Launch gate	Legal pages with operator info; cookie consent; accessibility audit fixes; NDPA legal review (external); payment provider decision (external); Vercel Pro / VPS decision; domain + HTTPS; monitoring dashboards; production build	Final audit checklist from the specification all green or explicitly waived in writing	2-3 wk

Table 16.1 - Phased implementation plan

Two sequencing decisions deserve explanation. Mobile is Phase 4, not Phase 1, because the web platform must exist for the mobile app to talk to, and the phone-to-phone fullscreen mode it depends on is only meaningful once live output works - this matches the approved-build-order guidance that web-first is the honest scope for a small team. Kaggle workers are not a phase item at all: they are optional dev-tier workers that may appear in P1's development environment if useful, constrained to non-commercial experimentation, with zero architectural footprint beyond the worker protocol they already speak.

Honest limitation register

Limitation	Impact	Design response
Mobile apps do not accept third-party virtual cameras (OS-level, verified)	No direct in-app integration with WhatsApp/Instagram/TikTok/Telegram/Facebook calls	Phone-to-phone physical adapter as a first-class output mode; copy never claims direct integration
MiniMax H3 self-hosting is not commercially licensed without written permission	Cannot run own H3 GPU fleet for production	H3 via official API (metered, legitimate); capability registry keeps future models swappable
Cold-start latency on sleeping GPU workers	First seconds of a session take tens of seconds to minutes to start after idle	Honest loading states from real backend state; never claimed instant
LiveKit Cloud free tier and R2 free tier are finite	Growth exhausts free allowances	Self-host LiveKit on VPS option; R2 paid tier is cheap at \$0.015/GB; both are config-level switches
Payment collection in Nigeria unverified	No automated charging until a provider is chosen and integrated	Credits granted manually; refunds stated as credit adjustments; provider decision at approval gate
App-store review is outside our control	Mobile ship dates are estimates only	Web platform remains fully functional without the app stores
Legal/NDPA review is external	Launch blocked until counsel signs off	Engineering commitments (data map, consent system, deletion flows) built to make that review fast

Table 17.1 - Platform limitations stated plainly (per the never-hide-limitations rule)

Risk register

Risk	Likelihood	Mitigation / fallback
H3 API pricing/limits change or access is restricted	Medium	Model is a capability-registry entry; swap to a verified alternative model without architecture change
RunPod availability/price shifts	Medium	Worker protocol is provider-independent; another GPU host (or own VPS) registers identically
LiveKit operational burden under self-hosting	Low-Med	Move to LiveKit Cloud; or Cloudflare Realtime fallback (verified pricing) behind the same client interface
Scope creep toward fake features to look complete	High (process risk)	This report's status vocabulary is the contract; phase exits require TESTED status; no mock counters or simulated states anywhere
Solo-build bandwidth	High	Vertical-slice phasing keeps the system demonstrably alive at every boundary; external items (legal, stores, payments) run in parallel, not in series
Prompt injection through user content	Medium	Data-not-instruction framing, tool allowlist, schema validation, policy veto, audit trail (Ch. 11)

Table 17.2 - Top project risks and mitigations

The fallback philosophy across every row is the same: each external dependency sits behind an interface that was designed for replacement - workers behind the worker protocol, models behind the capability registry, media infrastructure behind LiveKit's client-facing semantics with Cloudflare Realtime documented as the alternate path, hosting behind Next.js portability. Nothing in the architecture treats any provider as permanent, which is the engineering expression of the specification's rule that providers are workers and capabilities, never the architecture itself.

18. Approval Gate and Open Decisions

This report is the specification's mandated stop point: audit and research complete, architecture proposed, costs and risks stated - and no major implementation begun. The build proceeds only after the decisions below are made. Each item lists the report chapter that argues it, so the approval can be specific rather than general.

#	Decision	Default if approved as-proposed
1	Approve the three-plane architecture and the technology stack of Chapter 5 (Next.js, Better Auth, Drizzle/Neon Postgres, pgboss, LiveKit, R2, RunPod workers, H3 via official API, PostHog)	P0 begins on approval
2	Approve the phased plan and its exit criteria (Chapter 16), including web-first with mobile at Phase 4	Phases run in order; each exits on TESTED status
3	Confirm H3-via-official-API as the production model path, accepting metered API cost and no self-hosted H3 for production (Ch. 3)	Capability registry ships with the H3 API adapter
4	Confirm phone-to-phone as the sole social-app integration story, with no direct-integration claims (Ch. 3, 9)	Marketing and docs copy reflects this exactly
5	Defer payment provider selection to Phase 5, with manual credits until then (Ch. 6)	No charging before integration; refunds stated as credit adjustments
6	Note the external, non-code items that gate launch: Apple \$99/yr + Google Play \$25 accounts, domain, NDPA legal review, hosting tier decision (Ch. 6, 12, 15)	These run in parallel during P3-P5, owned by you

Table 18.1 - Decisions required to proceed

Upon approval, the first action is Phase 0: repository scaffolding, the Drizzle schema for users, sessions, jobs, workers, consent, and audit, Better Auth integration with the RBAC role set, the .env.example, and the CI pipeline - committed in small, reviewable units with tests from the first commit. Until then, per the specification's own final instruction, nothing is built. The audit is complete, the research is verified to the extent current documentation allows, the architecture is proposed with its reasoning exposed, and the risks, costs, and limitations are on the table rather than under it. The gate is yours.